Bloque 06 — Guía práctica de Observabilidad en Cloud

Monitoreo · Dashboard · Auto-reparación · HPA · Alarmas con correo (Semana 14)

Noviembre 2026

Contenido

Guía práctica — Bloque 06: Observabilidad en Cloud (Semana 14) 1

Guía práctica — Bloque 06: Observabilidad en Cloud (Semana 14)

> **Objetivo:** convertir el laboratorio en una sala de control. Vas a monitorear el microservicio, verlo en un dashboard de CloudWatch, provocar fallas reales y carga creciente para observar en vivo la auto-reparación, el autoescalado (HPA) y las alarmas llegando a tu correo.
>
> **Duración:** 45–60 min · **Dónde:** contenedor `devops-eks-lab` (`cd /root/work/bloque06-observabilidad` · **Dashboard:** se abre en el navegador de tu Mac

---

Prerrequisito: la app debe estar corriendo

Todo este bloque asume que los pods de `ep02` están `Ready`:

```
kubectl get pods -n ep02
```

Estado	Qué hacer
Todos `Running`/`Ready`	Continúa a la Fase 1
`ImagePullBackOff`	Ejecuta `push-imagenes.sh` desde tu Mac (imágenes en ECR) y espera 2–3 min

---

Fase 1 — Monitoreo simple: la métrica personalizada

Concepto (1 minuto)

CloudWatch recibe métricas de dos formas: **automáticas** (EC2, ELB, EKS ya publican las suyas) y **push** (tu aplicación las envía con `put-metric-data`). El script `push-metrica.sh` publica el namespace personalizado `Ep02/Microservicio` con dos números sobre el backend:

- `PodsActivos` — cuántos pods Ready tiene el backend
- `SaludBackend` — `1` si `/actuator/health` responde, `0` si no

Práctica

1. Publica la métrica una vez y mira el resultado:

```
bash push-metrica.sh
```

2. Deja el monitoreo continuo en una terminal (es tu "pulso del sistema" durante toda la sesión):

```
watch -n 15 bash push-metrica.sh
```

> Abre **3 terminales del contenedor**: (A) `watch` de push-metrica, (B) para las demos, (C) libre. En tu Mac deja abierta la pestaña del dashboard (Fase 3).

Fase 2 — Preparar alarmas y correo (`ejecutar.sh`)

El script es idempotente: crea el topic SNS `ep02-alertas`, publica la métrica por primera vez y crea 4 alarmas.

```
bash ejecutar.sh
```

El correo de alertas: el script lee automáticamente la variable `alertemail=` del archivo `secrets.txt` (raíz del laboratorio) y suscribe ese correo al topic. No te pregunta nada si la variable está definida.

```
secrets.txt (raíz del laboratorio)
----------------------------------
alertemail=tu-correo@duoc.cl
```

> Confirma la suscripción:** llega un correo de AWS Notifications con un link "Confirm subscription". Sin confirmar, las alarmas cambian de estado pero **no llegan correos**.

Al final verás el estado inicial: todas las alarmas en `OK` (o `INSUFFICIENT_DATA` si la métrica aún no tiene datos — corre `push-metrica.sh` de nuevo).

Fase 3 — El Dashboard: tu sala de control

Crea el dashboard personalizado en CloudWatch:

```
bash crear-dashboard.sh
```

El script detecta solo el LoadBalancer, la instancia EC2 y las alarmas `ep02-*` existentes, e imprime una URL. **Ábrela en el navegador de tu Mac** (el contenedor no tiene navegador). También queda en la consola: *CloudWatch -> Dashboards -> ep02-observabilidad*.

Qué muestra cada widget:

Widget	Fuente	Qué observarás
SaludBackend (0/1)	Métrica personalizada	Caerá a 0 en la Demo 1
PodsActivos	Métrica personalizada	Sube/baja con réplicas del backend
CPU del nodo	`AWS/EC2`	Sube en la Demo 2 con la carga
ELB: requests y 5XX	`AWS/ELB`	Tráfico real y errores
ELB: hosts saludables	`AWS/ELB`	Cuántos nodos reporta el ELB
Estado de las alarmas	Widget de alarmas	Verde -> rojo en vivo

---

Fase 4 — Demo 1: matar pods y verlos recrearse (auto-reparación)

Concepto

El ReplicaSet del Deployment es una **promesa** de N réplicas. Borrar un pod no es un error: es un evento que Kubernetes atiende solo. En producción esto es idéntico a cuando un nodo muere.

Práctica

```
bash demo-1-auto-reparacion.sh
```

El script: (1) verifica que todo esté sano, (2) borra 1 pod del backend y 1 del frontend, (3) muestra la línea de tiempo `T+0s, T+5s...` hasta que ambos vuelvan a `Ready`.

Mira el dashboard mientras corre y registra:

- El **frontend** (nginx) se recupera en ~5–15 s: su readiness probe es inmediata.
- El **backend** tarda ~60–90 s: su `readinessProbe` tiene `initialDelaySeconds: 50` — puede estar vivo y aun así "no listo". Así protege el balanceo.
- `SaludBackend` cae a `0` y la alarma `ep02-backend-salud` pasa a **ALARM** (~1–2 min después).
- Con el `watch` de la terminal (A) verás `PodsActivos` caer y volver.

Evidencia: captura de la línea de tiempo + captura del dashboard con la alarma en rojo + tiempo de recuperación de cada deployment.

Fase 5 — Demo 2: carga que tiende a crecer (HPA + alarmas)

Concepto

El HPA escala réplicas según CPU promedio del pod (backend: 1->10 al 70%; frontend: 2->6 al 60%). Escala rápido hacia arriba; al bajar espera ~5 min de estabilización.

Práctica

```
# Carga estándar: 8 procesos HTTP durante 5 minutos
bash demo-2-prueba-carga.sh

# Variantes:
DURACION=180 WORKERS=6 bash demo-2-prueba-carga.sh   # más corta y suave
WORKERS=12 DURACION=420 bash demo-2-prueba-carga.sh  # más agresiva
ESPERAR_BAJADA=1 bash demo-2-prueba-carga.sh         # quedarse a ver el downsizing
```

La carga dispara peticiones HTTP reales contra `http://<LB>/ep02/actuator/health` y cada 15 s imprime el estado de los HPA y la CPU.

Invitación a mirar el dashboard durante la demo:

1. **Minuto 0–1:** la CPU del nodo empieza a subir (widget EC2) y el volumen de requests crece (widget ELB).
2. **Minuto 1–3:** `TARGETS` del HPA backend cruza 70% -> `REPLICAS` pasa de 1 a 2, 3… (verifica en paralelo: `kubectl get hpa -n ep02 -w`). En el dashboard verás `PodsActivos` escalar.
3. **Si saturas el backend:** el ELB acumula 5XX (widget ELB) y salta la alarma `ep02-elb-5xx`.

4. **Al terminar la carga:** las réplicas **no bajan de inmediato** — el HPA espera su periodo de estabilización (~5 min). Con `ESPERAR_BAJADA=1` el script se queda presenciando la bajada.
5. **Nodo saturado:** si la CPU del t3.large cruza 60% sostenida 2 min, salta `ep02-cpu-nodo-alta` — distingue: esa alarma es del **nodo** (infraestructura), no de los pods.

Evidencia: tabla resumen que imprime el script (`reports/demo2-carga-*.md`), `kubectl get hpa` antes/durante/después, y el dashboard con la CPU y las réplicas subiendo.

Fase 6 — Alarmas: qué se configuró y cómo probarlas

Las 4 alarmas (creadas en Fase 2)

Alarma	Métrica	Condición	Prueba que la dispara
`ep02-backend-salud`	personalizada `SaludBackend`	< 1 por 1 min	Demo 1 (backend caído)
`ep02-backend-pods`	personalizada `PodsActivos`	< 1 por 1 min	Demo 1 (0 pods Ready)
`ep02-cpu-nodo-alta-*`	automática `AWS/EC2` CPUUtilization	≥ 60% por 2 min	Demo 2 (carga sostenida)
`ep02-elb-5xx`	automática `AWS/ELB` HTTPCode_ELB_5XX	≥ 5 por min	Demo 2 (backend saturado)

Práctica de verificación

1. **En el dashboard:** el widget de alarmas debe mostrar el ciclo `OK -> ALARM -> OK` en verde/rojo durante las demos.
2. **En la consola:** *CloudWatch -> Alarms* — revisa estado, historial ("State transitioned to ALARM…") y la gráfica de la métrica.
3. **En tu correo** (el de `alertemail=`): cuando una alarma salta llega un email de **AWS Notifications** con el nombre, el motivo y el timestamp; al recuperarse llega otro de "OK". Si no llegó nada: ¿confirmaste la suscripción? ¿revisaste spam?
4. **El ciclo de vida que debes poder explicar:** `INSUFFICIENT_DATA -> OK -> ALARM -> OK`. Una alarma sobre una métrica que nadie publica queda eterna en `INSUFFICIENT_DATA` — por eso el `watch` de la Fase 1 importa.

Evidencia: captura de la alarma en `ALARM` en la consola, captura del correo recibido, y el historial de transición de estado.

Fase 7 — (Opcional) Service mesh: Istio + Kiali

```
bash opcional-istio-kiali.sh
```

Instala el perfil demo de Istio (sidecar Envoy en los pods), Prometheus + Kiali, y un Gateway + VirtualService que exponen la app por la malla. Genera tráfico (Demo 2) y abre Kiali en tu Mac:

```
kubectl port-forward svc/kiali -n istio-system 20001:20001
# -> http://localhost:20001  (Traffic Graph)
```

Requiere que el contenedor haya levantado con `-p 20001:20001`. Limpieza: `LIMPIAR=1 bash opcional-istio-kiali.sh`.

Para tus conclusiones: la malla resuelve lo que ni el pipeline (reglas de calidad/seguridad: Sonar, Snyk, Trivy) ni CloudWatch (alarmas) resuelven — control del tráfico entre servicios: rutas, reintentos, mTLS, y visualización de errores servicio a servicio.

Limpieza

```
bash limpiar-bloque06.sh      # borra alarmas + topic SNS (las métricas expiran solas)
```

Al finalizar la semana, limpieza general del laboratorio: `../aws-limpiar.sh` (evita recursos cobrando tras la sesión Academy).

Solución de problemas

Síntoma	Causa probable	Qué hacer
Widgets de SaludBackend/PodsActivos vacíos	Nadie publica la métrica	Corre `watch -n 15 bash push-metrica.sh`
Alarmas en `INSUFFICIENT_DATA`	Ídem	Ídem — la métrica necesita datos
No llegan correos	Suscripción SNS sin confirmar	Busca el email de AWS Notifications y confirma
`kubectl top` sin datos	Metrics Server recién instalado	Espera 2–5 min
HPA no escala en Demo 2	Carga insuficiente o ya al objetivo	Sube `WORKERS=12` o `DURACION=420`
Alarma CPU del nodo no salta	t3.large reparte la carga	Válido reportarlo como observación
Dashboard no se crea	Permisos de la sesión Academy	Alarmas y métricas siguen en la consola
Pods `ImagePullBackOff`	ECR sin imágenes	`push-imagenes.sh` desde tu Mac

Resumen de evidencias para el informe

#	Evidencia	Se obtiene en
1	Salida de `push-metrica.sh` con la métrica publicada	Fase 1
2	Dashboard con todos los widgets con datos	Fase 3
3	Línea de tiempo de auto-reparación + alarma en rojo	Fase 4 (Demo 1)
4	HPA escalando réplicas + CPU del nodo subiendo	Fase 5 (Demo 2)

#	Evidencia	Se obtiene en
5	Alarma en `ALARM` en consola + correo SNS recibido	Fase 6
6	(Opcional) Traffic Graph de Kiali	Fase 7

Preguntas de reflexión: ¿cuánto tardó en recuperarse cada deployment y por qué difieren? ¿Qué diferencia hay entre escalar pods y saturar un nodo? ¿En qué momento del ciclo de vida actúa una regla de pipeline (Snyk) vs una regla de malla (mTLS)?